

Bridge Notes — Python to FastAPI

Eleven gaps between what you were taught and what the API assignment assumes | Exercises, not reading

Do these before Stage 1. This is not a FastAPI tutorial. It is the Python underneath FastAPI, which your twelve days did not reach.

How to use this

Your Python schedule ran from W3Schools, and W3Schools stops one layer below the code you are about to write. It taught you syntax. FastAPI is built out of three Python features W3Schools never mentions, and Pydantic is built out of a fourth.

Nothing here is difficult. All of it is unfamiliar, which feels the same from the inside and is not the same thing at all. Every example uses your own Kaveri tables, so none of it is throwaway.

Work through it in order. Each item ends with an exercise. Do the exercise, because reading this document will not move anything into your hands.

Write `def`, not `async def`. The event loop appears nowhere in your twelve days, and a blocking database call inside an `async def` route stalls every other request to the whole API. FastAPI runs plain `def` routes in a thread pool and handles this for you correctly. If you want to write `async def` anyway, you must first be able to explain the previous two sentences to the person sitting next to you.

Part 1 — Python you need on day one

1. Type annotations

W3Schools taught you `x = 5`. FastAPI reads annotations at runtime and builds request validation, response serialisation and the entire Swagger page out of them. An annotation you type carelessly becomes somebody else's 422.

```
from datetime import date

def nights(check_in: date, check_out: date) -> int:
    return (check_out - check_in).days
```

You need four shapes: `list[Room]`, `dict[str, int]`, `str | None`, and `Decimal` for money. Never `float` for money, for the same reason your `nightly_rate` column is `NUMERIC` and not `REAL`.

Python itself does not enforce any of this. Nothing stops you passing a string to `nights()`. FastAPI is what enforces it, and only at the boundary.

Exercise. Annotate every function in your SQL mini-project CLI. Then deliberately pass a wrong type and confirm Python does not complain.

2. Decorators

`@app.get("/rooms")` is a decorator. You have never written one, and you are about to write forty.

```
def logged(fn):
    def wrapper(*args, **kwargs):
        print("calling", fn.__name__)
        return fn(*args, **kwargs)
    return wrapper

@logged
def total_paid(booking_id: int) -> Decimal:
    ...
```

A decorator is a function that takes your function and returns a replacement. `@app.get(...)` does not replace yours, it registers it in a routing table and hands it back unchanged. That is the whole trick.

Exercise. Write a decorator that times a function and prints the duration in milliseconds. Put it on a `psycpg2` query from your mini project. You have now built a crude version of what task 9.1 asks you to do properly.

3. Generators and `yield`

Day 4 covered iterators and stopped short of the thing you need. Every database dependency in FastAPI is a generator.

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`yield` hands the session to your endpoint and pauses there. Everything after `yield` runs once the response has been sent. That is how a connection returns to the pool even when your endpoint raises an exception halfway through.

Exercise. Write a generator that yields one row at a time from `SELECT * FROM bookings`. Put a `print` before the `yield` and another after the loop that consumes it. Predict the order of the output before you run it, then run it.

4. Enums

Rule 7 gives you five booking states. In the database they are text with a `CHECK` constraint. In Python they should be an `Enum`, so that a typo is an error at import rather than a 500 at 4pm on a Saturday.

```
from enum import Enum

class BookingStatus(str, Enum):
    confirmed = "confirmed"
    checked_in = "checked_in"
    checked_out = "checked_out"
    cancelled = "cancelled"
    no_show = "no_show"
```

Inherit from `str` as well as `Enum`, or it will not serialise to JSON and you will spend an hour finding out why.

Exercise. Write the enum, then write `can_transition(current: BookingStatus, target: BookingStatus) -> bool` implementing the state machine you drew in task 1.9. Keep the file. Task 5.4 needs it.

5. Packages and project layout

One `main.py` is where every tutorial begins and where no real project stays. You need this working before Stage 4, not during it.

```

app/
  __init__.py
  main.py
  db.py
  dependencies.py
  errors.py
  models/
  schemas/
  routers/

```

`from app.db import get_db` works only if `__init__.py` files exist and you run `uvicorn app.main:app` from the project root. Import errors here are the single most common way to lose an afternoon.

Exercise. Split your SQL mini-project CLI into a package with at least three modules. Get the imports working from a clean shell. If you cannot, fix it now.

6. Environment variables and secrets

Task 2.10 requires your application to refuse to start when `SECRET_KEY` is absent. There is a one-character difference between doing that and not doing it.

```

import os

SECRET_KEY = os.environ["SECRET_KEY"] # raises at import if missing. Correct.
SECRET_KEY = os.getenv("SECRET_KEY")  # silently None. Your JWTs are now signed with "None".

```

Use `python-dotenv` to load a `.env` file in development. Commit `.env.example` with the keys and no values. Put `.env` in `.gitignore` before your first commit, not after.

Exercise. Create all three files. Make one commit, then run `git log -p | grep -i secret` and confirm nothing comes back.

Part 2 — Concepts you need by Stage 3

7. HTTP status codes

Stage 3 asks you to decide a status code for every failure in the system, and marks you on it. These are the eight you will use.

Code	What it means
200	Here is what you asked for.
201	I created something. Include its location or its id.
204	Done, and there is nothing to send back.
400	Your request is malformed. I could not even parse it properly.
401	I do not know who you are. Your token is missing, expired or forged.
403	I know exactly who you are, and you may not do this.
404	There is no such thing. Or there is, and you are not allowed to know that.
409	Your request is fine and conflicts with the state of the world. The room is taken.
422	Your request parsed, and a field is wrong. Guest count of minus two.

The line between 401 and 403 is identity versus permission. The line between 400 and 422 is parseable versus sensible. The line between 404 and 403 is a decision you have to make deliberately, and task 4.3 makes you defend it.

Exercise. Write those nine down from memory. Check yourself. You will use every one of them.

8. Pydantic, and where it stops

There are two incompatible Pydantic versions in circulation and tutorials mix them freely. You are using v2: `field_validator`, `model_config`, `model_dump()`. If a Stack Overflow answer says `@validator` or `.dict()`, it is v1 and it will not run. Pin the version in `requirements.txt` on day one.

More important is the boundary. Pydantic validates the shape of a request. It cannot validate the truth of one.

Pydantic can tell you	Only the database can tell you
<code>check_out</code> is not a date	Room 101 is already booked those nights
<code>guests</code> is not an integer	This room type allows two guests, not four
<code>rating</code> is outside 1 to 5	This guest already reviewed this stay
<code>email</code> is not shaped like an email	This email is already registered

Everything in the right column is a constraint you already wrote. Do not re-implement it in Python. Task 8.12 exists to catch you if you do.

Exercise. Write a `BookingCreate` model with a check-in date, a check-out date and a guest count. Add a validator rejecting `check_out <= check_in`. Then write one sentence explaining why the database needs that same rule anyway.

Part 3 — Tooling you need by Stage 9

9. pytest

Testing appears nowhere in your seventeen days. Task 9.5 requires a suite, so learn it here on something small.

- Files named `test_*.py`, functions named `test_*`. Plain `assert`, no assertion methods.
- `@pytest.fixture` for setup. Shared fixtures go in `conftest.py`.
- FastAPI's `TestClient` calls your app in-process, with no server running.
- `@pytest.mark.parametrize` turns the fourteen attacks in Stage 8 into one test function.
- `pytest --cov=app` reports coverage.
- Wrap each test in a transaction and roll it back at the end, or your tests will slowly destroy the seed data you spent Stage 3 building last time.

```
def test_zero_nights():
    with pytest.raises(ValueError):
        nights(date(2026, 3, 1), date(2026, 3, 1))
```

Exercise. Write four tests for the `nights()` function from item 1, including the zero-night case and one where check-out precedes check-in. Then write one `TestClient` test against a hello-world FastAPI app.

10. Two requests at the same time

Task 8.19 needs two genuinely concurrent requests, and Postman's runner cannot produce them. Three ways, none of which need `async`:

```
import threading, httpx

def book():
    r = httpx.post(URL, json=PAYLOAD, headers=H)
    print(r.status_code, r.json())

t1, t2 = threading.Thread(target=book), threading.Thread(target=book)
t1.start(); t2.start(); t1.join(); t2.join()
```

Or two terminals with `curl` and fast fingers. Or `printf '%s\n' 1 2 | xargs -P 2 -I{} curl ...`. What matters is that both requests are in flight at the same moment. Firing one after the other proves nothing, and it is exactly the mistake that let the original spreadsheet double-book a room.

Exercise. Build a FastAPI endpoint that sleeps for three seconds and returns the current time. Hit it twice in parallel and prove from the two timestamps that the calls overlapped.

11. Rate limiting

Task 9.3 asks you to rate limit login. The idea is one counter per key per time window: how many requests has this IP address made to this endpoint in the last minute.

A dictionary in memory is ten lines and is the right place to start. It is also wrong the moment you run two `uvicorn` workers, because each worker has its own dictionary and your limit of five silently becomes ten. Production answers put the counter in Redis. `slowapi` wraps this for FastAPI.

Exercise. Write the in-memory version. Then write two sentences on what breaks with two workers, and keep them for 9.3.

Readiness check

You are ready for Stage 1 when every row here is true. If any of them is not, the task in the right column is where you will stall.

You can	Otherwise you stall at
Read and write a type annotation, including optional types	Every endpoint you write, from 4.1 onward
Write a decorator from scratch and explain what <code>@app.get</code> does	Stage 4, immediately
Explain what runs after <code>yield</code> in <code>get_db</code> and when	5.12, the transaction boundary
Model your five booking states as an <code>Enum</code>	5.4, the state machine
Run a multi-module package from a clean shell	Stage 4, and every stage after it
Make an app fail loudly on a missing environment variable	2.10, and the Fail row of the marking scheme
Recite 401 versus 403 versus 404 without hedging	3.8, worth marks on its own
Say what Pydantic validates and what only Postgres can	8.12, and the Weak signal row
Write a pytest fixture and a parametrised test	9.5
Fire two HTTP requests that genuinely overlap	8.19, the concurrency test

Ten of these eleven items are one page of reading and twenty minutes of typing. The eleventh is pytest, which is a habit rather than a topic, and the only one worth spending a full day on.